The http module & createServer

10 min read · Updated 24 Jul 2026 · Topic 30 of 70

Bookmark · Mark done

Why it matters

Every web framework you have ever used in Node — Express, Fastify, Koa, Next.js, NestJS — is, at the bottom, a thin wrapper around one built-in module: `node:http`. When a request lands on your server, it is `http.Server` that accepted the TCP connection, parsed the bytes, and handed your code a `req` and a `res`. Frameworks add routing, middleware, and ergonomics, but they never replace this layer; they sit on top of it.

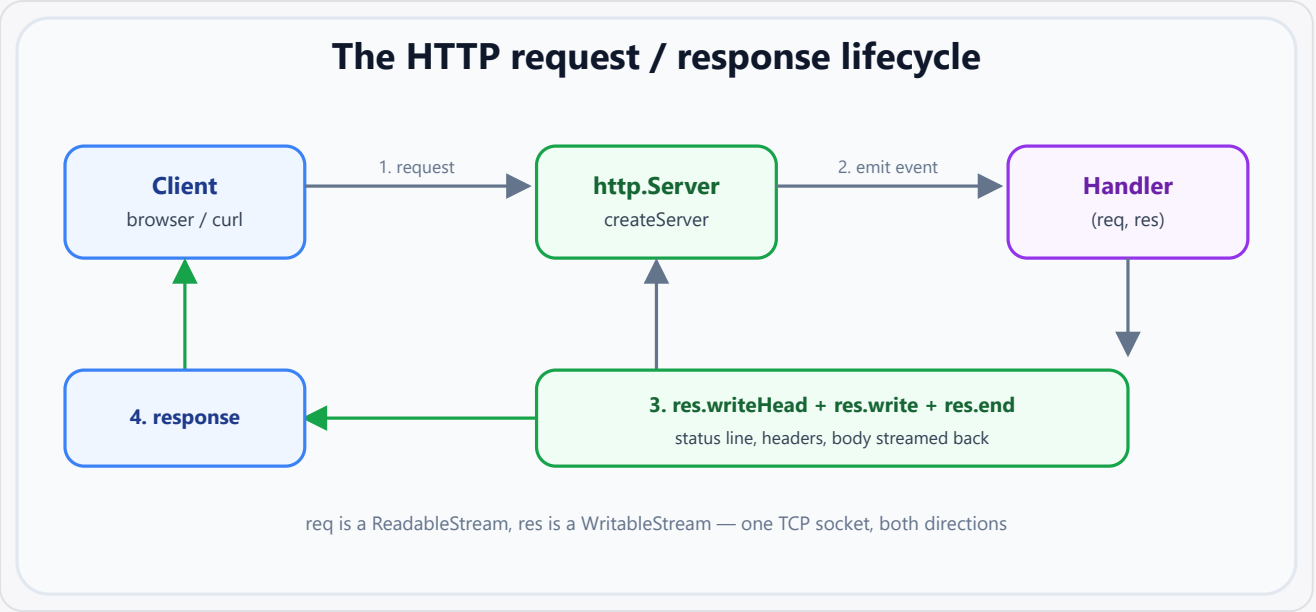
Understanding the raw module pays off in three concrete ways. First, **debugging**: when a request hangs, a socket leaks, or a response never flushes, the symptoms surface at the `http` layer, and you cannot reason about them if it is a black box. Second, **performance**: keep-alive, header size limits, and streaming all live here, and the defaults matter under load. Third, **interviews**: "build an HTTP server without Express" is one of the most common Node screening questions precisely because it reveals whether you understand the platform or just an abstraction over it.

This topic strips everything away. No `npm install`, no framework — just the module that ships with Node, the `createServer` factory, and the lifecycle of a single request from socket to response.

The concept, simply

An HTTP server is a long-running process that listens on a TCP port. When a client opens a connection and sends an HTTP request, the server runs a function you supplied, giving it two objects: a *request* (what the client sent) and a *response* (a handle you write the reply into). You never poll or block waiting for requests — Node's event loop calls your function each time one arrives.

Two ideas make this click. The server is an **EventEmitter**: the callback you pass to `createServer` is registered as a listener for the `'request'` event. And `req` / `res` are **streams** — `req` is readable (the incoming body arrives in chunks) and `res` is writable (you push bytes back out). Once you see the request/response pair as "an event carrying two streams," the rest of the module stops being magic.



The request/response lifecycle: one TCP socket, req is readable, res is writable.

How it works

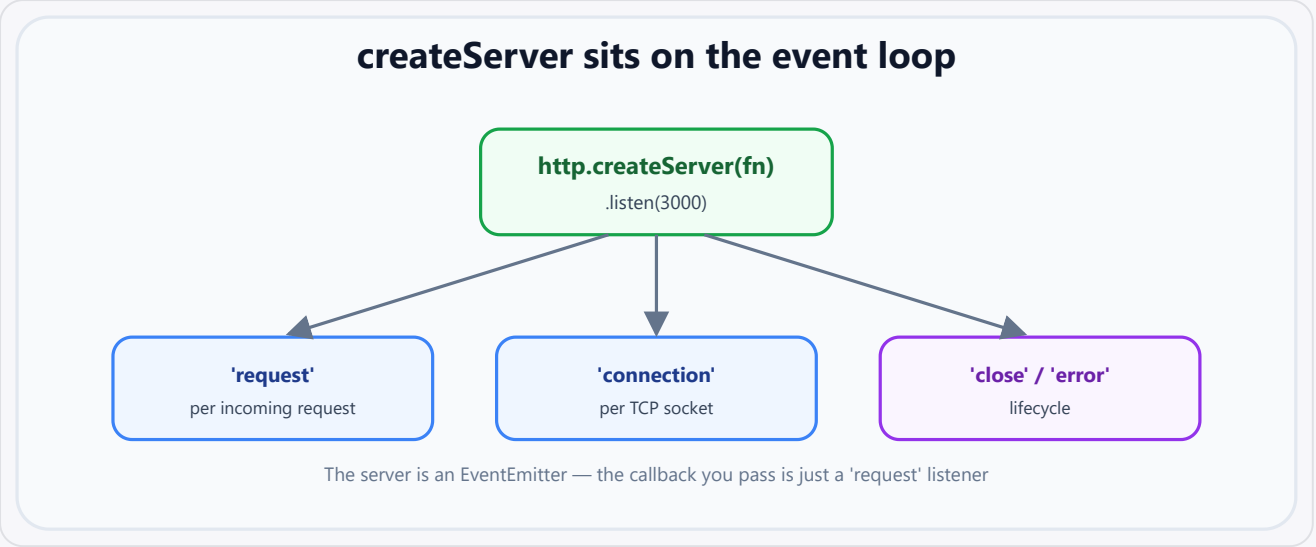
`http.createServer([options][, requestListener])` returns an `http.Server` instance. Calling `server.listen(port)` binds a listening socket and, from then on, the OS hands new TCP connections to Node. For each connection Node reads bytes off the socket and feeds them into an internal C++ HTTP parser (llhttp). The parser recognises the request line and headers, and the moment it has a complete header block it constructs an `http.IncomingMessage` (your `req`) and an `http.ServerResponse` (your `res`) and emits `'request'`.

Crucially, the `'request'` event fires **after headers are parsed but before the body is read**. The body may still be arriving. That is why `req` is a readable stream: you subscribe to its `'data'` and `'end'` events (or pipe it) to consume the payload. For a `GET` with no body, `'end'` fires almost immediately.

The `res` object is a writable stream with HTTP semantics layered on. Writing works in three logical phases: (1) the **status line and headers** — set with `res.statusCode`, `res.setHeader()`, or all at once with `res.writeHead()`; (2) the **body** —

before the first byte of body is written; after that they are “sent” and mutating them throws `ERR_HTTP_HEADERS_SENT`.

The server itself emits more than `'request'`. It emits `'connection'` for each new TCP socket, `'close'` when it stops, `'clientError'` when a malformed request cannot be parsed, and `'listening'` once the port is bound. Because it is an EventEmitter, `createServer(fn)` is literally sugar for `new http.Server().on('request', fn)`.



The server is an EventEmitter; your callback is just a 'request' listener.

One more mental model that avoids bugs: the callback runs **once per request**, and there is no global “current request” — every request has its own `req` / `res` closure. State you want to share across requests lives in module scope; state for a single request lives in that invocation.

It also helps to know that one TCP socket can carry **many** requests. Because HTTP/1.1 keep-alive is on by default, after you call `res.end()` Node keeps the connection open and the llhttp parser simply resets to await the next request line on the same socket, emitting `'request'` again with a brand-new `req` / `res` pair. So the `'connection'` event fires once per socket, but `'request'` can fire repeatedly on that same socket. This is why you attach per-request logic in the `'request'` listener and per-connection logic (rate-limiting by IP, socket timeouts) in `'connection'`. The pairing between a request and its response is positional: HTTP/1.1 has no request IDs, so responses must be

Finally, the server exposes a set of timeouts that are your first line of defence against slow or abusive clients. Node v18+ ships with safe defaults, but you should still set them deliberately for a production service. The table below summarises the ones that matter most:

Property	Default (v20/22)	What it guards
`server.headersTimeout`	60000 ms	Max time to receive the complete header b
`server.requestTimeout`	300000 ms	Max time to receive the entire request (he
`server.keepAliveTimeout`	5000 ms	How long an idle keep-alive socket waits fo
`server.timeout`	0 (off)	Inactivity timeout on the socket once a rec
`server.maxHeadersCount`	2000	Caps the number of headers to bound par

Code examples

The smallest complete server. It answers every request with plain text.

JAVASCRIPT

Copy

```
const http = require('node:http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain; charset=utf-8');
  res.end('Hello from raw Node
');
});

server.listen(3000, () => {
  console.log('Listening on http://localhost:3000');
});
```

The same server written to show that the callback is just an event listener, plus lifecycle hooks you will use in production (graceful logging, error handling):

JAVASCRIPT Copy

```
const http = require('node:http');

const server = http.createServer();

server.on('request', (req, res) => {
  console.log(req.method, req.url);
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('ok');
});

server.on('clientError', (err, socket) => {
  // Malformed request line/headers — parser could not build a req.
  socket.end('HTTP/1.1 400 Bad Request\r
\r
');
});

server.on('listening', () => console.log('up'));
server.listen(3000);
```

Streaming a response instead of buffering it — this is where `res` being a writable stream earns its keep. We send a large body in chunks without ever holding it all in memory:

JAVASCRIPT Copy

```
const http = require('node:http');

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  let n = 0;
  const timer = setInterval(() => {
    res.write(`line ${++n}
`);
    if (n === 5) {
      clearInterval(timer);
      res.end('done
```

```
  // If the client disconnects mid-stream, stop working.
  res.on('close', () => clearInterval(timer));
}).listen(3000);
```

Graceful shutdown — a real service must stop accepting new connections but let in-flight requests finish. `server.close()` does exactly that:

JAVASCRIPT Copy

```
const server = http.createServer(handler).listen(3000);

process.on('SIGTERM', () => {
  server.close(() => {
    console.log('drained, exiting');
    process.exit(0);
  });
  // Force-exit if connections don't drain in time.
  setTimeout(() => process.exit(1), 10_000).unref();
});
```

Common mistakes & gotchas

- **Forgetting `res.end()`.** If you never call `end()`, the response never finishes, the client hangs until timeout, and the socket is held open. Every code path — including error branches — must terminate the response exactly once.
- **Setting headers after the body.** `res.setHeader()` or `writeHead()` after the first `res.write()` throws `ERR_HTTP_HEADERS_SENT`. Decide status and headers first.
- **Assuming the body is available in the callback.** The `'request'` event fires before the body arrives. Reading `req.body` gives `undefined` — there is no such property on the raw object; you must consume the stream.
- **Blocking the event loop.** A synchronous CPU-heavy loop or `fs.readFileSync` inside the handler stalls *every* concurrent request, because one thread services them all.

- **Ignoring backpressure.** `res.write()` returns `false` when the kernel buffer is full; ignoring it while pumping a fast source balloons memory. Use `pipe()` or honour the return value.
- **Leaking the port in tests.** Forgetting `server.close()` keeps the process alive and the port bound, causing `EADDRINUSE` on the next run.

Best practices

- **Import with the `node:` prefix** (`require('node:http')`) so it is unambiguous that you mean the builtin, not an npm package.
- **Always attach a `'clientError'` handler** and an `'error'` handler on the server — unhandled server errors crash the process.
- **Set explicit timeouts** (`server.requestTimeout`, `server.headersTimeout`, `server.keepAliveTimeout`) to defend against slow-loris style attacks; the defaults are safer in v20 but still worth setting deliberately.
- **Terminate the response in a `finally`-style single exit** so no branch forgets `res.end()`.
- **Implement graceful shutdown** with `server.close()` on `SIGTERM` for zero-downtime deploys.
- **Never trust request input** at this layer — size-limit the body and validate headers before acting on them.

⭐ Interview lens

Q: What does `http.createServer(fn)` actually do with `fn`?
It constructs an `http.Server` (an EventEmitter) and registers `fn` as a listener for the `'request'` event. It is equivalent to `new http.Server().on('request', fn)`. The function is invoked once per incoming HTTP request with a fresh `req` / `res` pair.

Q: When exactly does the request callback fire — before or after the body arrives?

`'data'` / `'end'` (or pipe it) to get the body. A common trap answer is "when the whole request is in" — wrong.

Q: Why can't you call `res.setHeader()` after `res.write()`?
Because `write()` flushes the status line and headers to the socket first; once sent, they are immutable on the wire. Node enforces this by throwing `ERR_HTTP_HEADERS_SENT`. Headers must be finalised before any body byte.

Q: Is `req` a stream? What kind?
Yes — `req` (an `IncomingMessage`) is a *readable* stream; `res` (a `ServerResponse`) is a *writable* stream. This is the single most important fact about the module and unlocks streaming uploads/downloads.

Q: Trap — if a request handler runs a 2-second synchronous loop, what happens to other clients?
They all wait. Node handles requests on one event-loop thread; a synchronous block freezes the entire process, so every concurrent request is delayed by 2 seconds. The fix is to make the work async or offload it (worker thread / child process).

Q: How do you shut a server down without dropping in-flight requests?
Call `server.close(callback)`: it stops accepting new connections and invokes the callback once existing ones drain. Pair it with a forced-exit timeout so a stuck keep-alive connection cannot block shutdown forever.

Cheat-card

- `const http = require('node:http')` — builtin, no install.

- `req` = `IncomingMessage` (readable); `res` = `ServerResponse` (writable).
- Callback fires after headers, before body. Body via `'data'` / `'end'`.
- Order: status + headers → `res.write()` → `res.end()`. End exactly once.
- Headers are immutable after first body byte (`ERR_HTTP_HEADERS_SENT`).
- Graceful stop: `server.close()` on `SIGTERM`.
- One thread services all requests — never block it.

QUICK CHECK — no marks, just for you.

1. What does http.createServer(fn) actually do with fn?

- ◯ It runs fn once immediately to initialise the server
- ◯ It replaces Node's built-in HTTP parser with fn
- ◯ It calls fn on a new thread for every TCP packet received
- ◯ It constructs an http.Server (an EventEmitter) and registers fn as a listener for the "request" event

Check answer

2. When does the "request" callback fire relative to the request body arriving?

- ◯ After the request line and headers are parsed, but generally before the body is fully received
- ◯ Only after the entire request, including the full body, has arrived
- ◯ Before any bytes are read from the socket
- ◯ After the response has been sent back to the client

3. Why does calling res.setHeader() after res.write() throw ERR_HTTP_HEADERS_SENT?

- ◯ setHeader can only be called once per response, regardless of write()
- ◯ The first write() flushes the status line and headers to the socket, and once sent they are immutable on the wire
- ◯ write() deletes all previously set headers, so there is nothing to modify
- ◯ Headers can only be set inside writeHead(), never with setHeader()

Check answer

‹ Previous
Backpressure, pipe & pipeline

Next ›
Parsing requests & writing responses

LEARN MORE

ARTICLES & DOCS

- HTTP | Node.js Documentation
  nodejs.org
- Introduction to Node.js | Node.js Learn
  nodejs.org
- Overview of HTTP | MDN
  developer.mozilla.org

VIDEOS





How to Create a Simple HTTP Server in Node.js | HTTP Module Basics

Node.js Backend Kit · Knowledge

Hand-picked references for this lesson.